30. B Trees

T.V. GEETHA

Welcome to the e-PG Pathshala Lecture Series on Data Structures. In this module we will discuss an important multi-way search tree – B-Trees which is commonly used for Database systems where data is stored externally on disks.

Learning Objectives

- To understand the concept of B Trees
- To discuss insertion into B Trees
- To describe deletion from B Trees
- To discuss the use B an B+ Trees for Indexing

30.1 Introduction to B Trees

B-Trees are general multi-way search trees commonly used in database systems or other applicationswhere data is stored externally on disks and keeping the tree shallow is important. In this scenario, each node takes one full {*page*, *block*, *line*} of

memory. The simpler B Trees we have already seen are of two types. 2-3 trees can be considered as a B tree of order 3 where each non-root node has either 2 or 3 subtrees. 2-3-4 trees can be considered as a B-tree order 4. Here each node can have 2, 3 or 4 subtrees.

30.2 Properties of B-Tree

A B-Tree T is a rooted tree having the following properties:

1. Every node x has the following fields:

    1. n[x] which indicates the number of keys currently stored in node x.
    2. The n[x] actual keys themselves, stored in non-decreasing order, so that key1[x] ≤ key2[x]..... ≤keyn-1[x] ≤ keyn[x][x].
    3. Leaf[x], a Boolean value that is TRUE if x is a leaf and FALSE if x is an internal node.

2. Each internal node x also contains n[x]+1 pointers (Children) c1[x], c2[x],... cn[x]+1[x].

3. The keys keyi[x] separates the range of keys stored in each subtree : If ki is any key stored in the subtree with root as ci[x] then k1 ≤ key1[x]≤ k2 ≤ key2[x] ≤ .....≤ kn[x][x] ≤ keyn[x]+1

4. All leaves have the same depth, which is the tree's height h.

30.2.1 Bounds on the number of keys of a node

There are lower and upper bounds on the number of keys that a node can contain: These bounds can be expressed in terms of a fixed integer t≥2 called **the minimum degree of B-Tree.**If every node other than the root must have at least t-1 keys, then root has at least t children. If the tree is non empty the root must have at least one key.Every node can contain at most 2t-1 keys. Therefore, an internal node can have at most 2t children. We say that a node is full if it contains exactly 2t-1 keys.For a **B-Tree of orderM**=2t, the number of keys in internal node isM-1 and the number of children of internal node is between M/2 and M. The Depth of B-Tree storing n items is O(logM/2N).

Each (non-leaf) internal node of a B-tree has between éM/2ù and M children and up to M-1 keys k1 <k2 < ... <kM-1(Figure 30.1). Children of each internal node are "between" the items in that node.

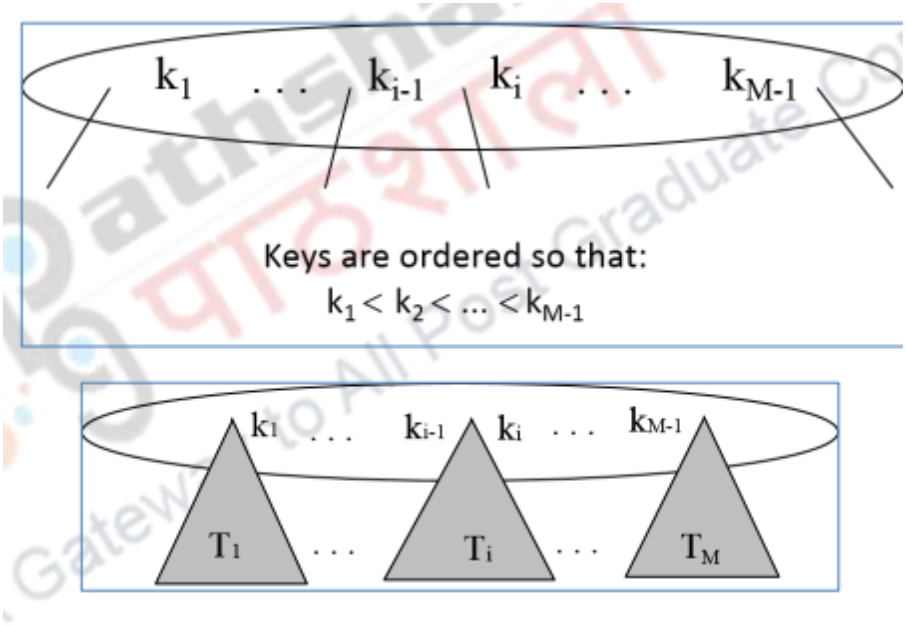

Figure 30.1 Bounds on the number of keys of a node

Suppose subtree Ti is the ith child of the node . Now all keys in Ti must be between keys ki-1 and ki , i.e. k i-1< Ti< ki where ki- 1 is the smallest key in Ti. All keys in first subtree T1< k 1and all keys in last subtree T M≥ kM- 1(Figure30.2).

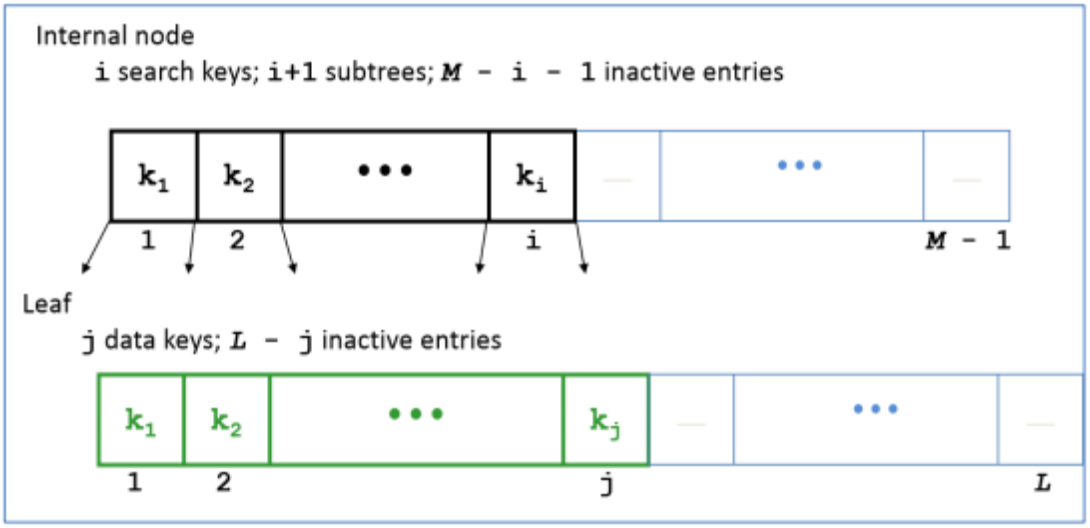

Figure 30.2 Entries in a B Tree

Example:

Let us consider a B-tree of order five that is m=5. The minimum number of entries is : Ceiling|5/2| – 1 = 2 entries while the minimum number of subtrees is Ceiling|5/2|=3. The maximum number of entries is 5 – 1 = 4 entries while the maximum number of subtrees is 5. An example of such a B tree is shown in Figure 30.3.

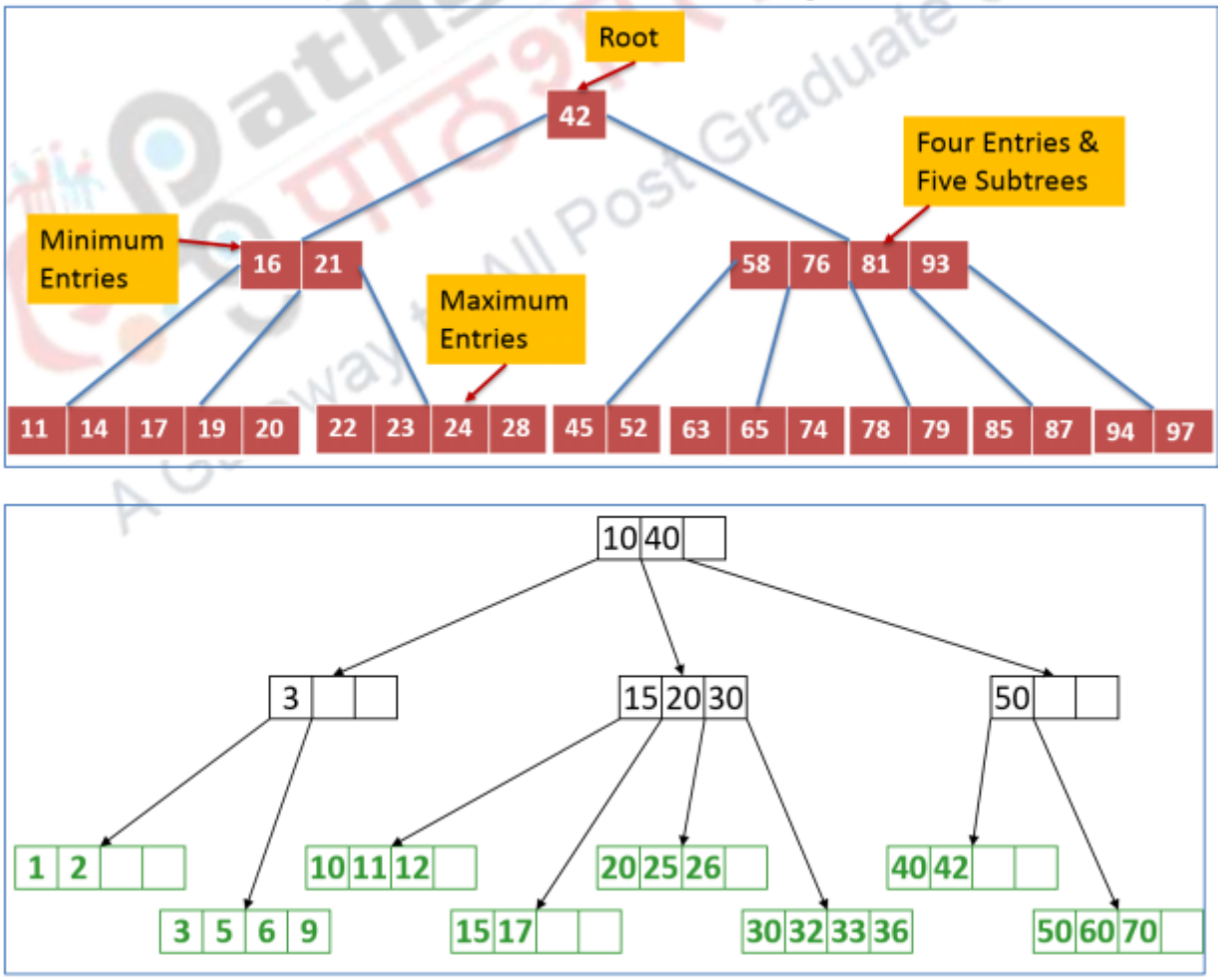

Figure 30.3 Examples of a B Tree

30.3 Insertion in B-Trees

The following are the steps for insertion. If after inserting the node into the appropriate sorted order, no internal node is over its key capacity (M-1) or in other words if the number of children of the internal node is less than or equal to M, the process is finished. However if some node has more than the maximum amount of child nodes then it is split into two nodes, each with the minimum amount of child nodes. This process continues recursively at the parent node. An overflow condition is said to occur when the root-node or a non-root node of a B-tree of order m overflows. This happens when after a key insertion, the root node contains m keys.

30.3.1 Insertion algorithm:

When we insert a data into the B tree we first search for the data item and then find at the appropriate leaf node . If the leaf node is not full, we insert the data into the leaf node. If leaf node is full, split leaf node and adjust parents up to root node . If any internal node overflows, we split it into two, and propagate the “middle” key to the parent of the node. If the parent overflows the process propagates upward. If the node has no parent, we need to create a new root node.

30.4 Deletion in B-Tree

Like insertion, deletion is also initiated at a leaf node. If the key to be deleted is not in a leaf, swap it with either its successor or predecessor (each will be in a leaf).The successor of a key k is the smallest key greater than k. In case we are using predecessor, the predecessor of a key k is the largest key smaller than k.In a B tree the successor and predecessor, if any, of any key is in a leaf node.

30.4.1 Simple Deletion:

30.4.1.1 Deletion of a key from the Leaf:

The removal of keys from the leaves can occur under two circumstances. One is when the key actually exists in the leaf of the tree . When we remove some key from the leaf, there can still be enough keys in the leaf so that there are (m/2-1) keys in total and the B Tree condition continues to be satisfied.

30.4.1.2 Deletion of a key from an internal node:

When the key exists in an internal node it must be moved to leaf by determining which leaf position contains the key closest to the one to be removed. In this case we need to locate the in-order successor of the key to remove and replace it with the key. After this step if the leaf node is in legal state (minimum capacity not violated) then the deletion has been completed. However if the inner node is in an illegal state then we need to redistributeits sibling node (a child of the same parent node) which can transfer one of its keys to the current node.

In some cases even when we c**oncatenate**the siblings of the node they do not have an extra key to share. In that case both these nodes are merged into a single

node (together with a key from a parent) and pointers updated accordingly. The process continues until the parent node remains in a legal state or until the root node is reached.

30.4.2 Underflow Condition

During deletion an underflow condition can also occur. A non-root node of a B-tree of order m underflows if, after a key deletion, it contains**ém / 2ù – 2** keys that is less than the minimum number of keys a node should have. The root node does not underflow. If it contains only one key and this key is deleted, the tree becomes empty.

30.4.3 Deletion algorithm:

The deletion algorithm works as follows:

If a node underflows, **rotate** the appropriate key from the **adjacent**right-orleft-sibling if the sibling contains at least **ém / 2ù**keys; otherwise perform a **merging**. Now let us discuss the five cases of deletion:

30.4.3.1 The Five Cases – Deletion in B Trees

1. The leaf does not underflow, then we just delete the key from the node.
2. The leaf underflows and the adjacent right sibling has at least ém / 2 ù keys then we **perform a left key-rotation.**
3. The leaf underflows and the adjacent left sibling has at least ém / 2 ù keys then we **perform a right key-rotation.**
4. The leaf underflows and each of the adjacent right sibling and the adjacent left sibling has at least ém / 2 ù keys then we p**erform either a left or a right key-rotation.**
5. The leaf underflows and each adjacent sibling has ém / 2ù – 1 keys then we **perform a merging**

30.5 Run Time Analysis of B-Tree Operations

Now we will discuss the run time analysis for B Tree operations. For a B-Tree of order M, each internal node has up to M-1 keys to search.Each internal node has

between (M/2) and M children and the depth of the B-Tree storing N items is $O(\log_{(M/2)}N)$

The maximum number of items in a B-tree of order M and height *h*: is as follows:

root $M - 1$

level 1 $M(M - 1)$

level 2 $M^2(M - 1)$

level h $M^h(M- 1)$

So, the total number of items is

$(1 + M+ M^2 + M^3 + \ldots + M^h)(M- 1) =[(M^{h+1} - 1)/ (M- 1)] (m - 1) = \mathbf{m^{h+1} - 1}$

When $m = 5$ and $h = 2$ this gives $53 - 1 = 124$. The run time is O(log M) to decide which branch to take at each node. But M the order of the B tree is small compared to N the total number of items. Therefore the total time to find an item is O(depth*log M) = O(log N).

30.6 Thinking about B-Trees

B-Tree insertion can cause (expensive) splitting and propagation. B-Tree deletion can cause (cheap) borrowing or (expensive) deletion and propagation. However propagation is rare if **M** and **L** are large. In addition repeated insertions and deletions can cause thrashing. For example if M = L = 128, then a B-Tree of height 4 will store at least 30,000,000 items

30.7 B-Tree Index

The B Tree is used as a standard index in relational databases. It allows for rapid tree traversal searching through an upside-down tree structure. Reading a single record from a very large table using a B-Tree index, can often result only in a few block readseven when the index and table are millions of blocks in size.Any index structure other than a B-Tree index is subject to overflow. Overflow is where any changes made to tables will not have records added into the original index structure, but rather tacked on the end.

30.8 Why B-Trees?

B-trees are suitable for representing huge tables residing in secondary memory because:

1. With a large branching factor m, the height of a B-tree is low resulting in fewer disk accesses.

2.The branching factor can be chosen such that a node conveniently corresponds to a block of secondary memory.

3. The most common data structure used for database indices is the B-tree. An **index** is any data structure that takes as input a property (e.g. a value for a specific field), called the search key, and ***quickly*** finds all records with that property.

4. B-Trees are always balanced.

5. B-Trees keep similar-valued records together on a disk page, hence takingadvantage of locality of reference.

6. B-Trees guarantee that every node in the tree will be full at least to a certain minimum percentage. This improves space efficiency while reducing the typical number of disk fetches necessary during a search or update operation.

30.9 B+Tree Characteristics

This tree is a variation of the B tree where the data records are only stored in the leaves. Here internal nodes store just keys. These keys are used for directing a search to the proper leaf. If a target key is less than a key in an internal node, then the pointer just to its left is followed. If a target key is greater or equal to the key in the internal node, then the pointer to its right is followed. B+ Tree combines features of ISAM (Indexed Sequential Access Method) and B Trees. If the B+ tree is implemented using disk, it is likely that the leaves contain key and pointer pairs where the pointer field points to the record of da ta associated with the key. This allows the data file to exist separately from the B+ tree, which functions as an “index” giving an ordering to the data in the data file. The B+ tree is the most widely used Index structure. Insert/delete at log F N cost; keep tree *height-balanced*. (F = fanout, N = # leaf pages). A minimum of 50% occupancy (except for root) is ensured. Each node contains **d**<= *m*<= 2**d** entries. The parameter **d** is called the *order* of the tree. B+ Tree supports equality and range-searches efficiently.

Summary

- Explained the concept of B Trees
- Discuss the insertion of B Trees
- Described deletion from B Trees
- Discussed the use B an B+ Trees for Indexing

---

you can view video on B Trees 

Web Links
http://peace.lakeheadu.ca/cs2412/slides/cs2412-s8.pdf
http://www.intelligence.tuc.gr/~petrakis/courses/datastructures/btrees.pdf
https://www.cs.purdue.edu/homes/bertino/348Spring2012/B-Trees%20and%20B+Trees.pdf
http://www.di.ufpb.br/lucidio/Btrees.pdf
http://www.geeksforgeeks.org/b-tree-set-1-introduction-2/
http://www.cs.nott.ac.uk/~psznza/G52ADS/btrees2.pdf
https://www.cs.usfca.edu/~galles/visualization/BTree.html
http://peace.lakeheadu.ca/cs2412/slides/cs2412-s8.pdf
Supporting & Reference Materials
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Adam Drozdek, "Data Structures and Algorithms in C++", Cengage; 4th edition,2013
Alfred V. Aho and Jeffrey D. Ullman, "Data Structures and Algorithms", 1983
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014